Distributed Multi-Model E-Commerce Analytics

MongoDB, HBase and Apache Spark — Technical Report

YOUR FULL NAME · YOUR STUDENT ID · AUCA Big Data Analytics, individual final project · Repository: https://github.com/KWIZERARENE/auca-bigdata-ecommerce

1. Overview and dataset

The project builds an analytics platform for a synthetic e-commerce business. Browsing behaviour and purchases are stored in two NoSQL systems chosen for their access patterns, cleaned and analysed with Apache Spark, and combined in an integrated funnel analysis that no single system could answer alone. All code, Docker setup and exact reproduction steps are in the repository README.

A seeded generator (seed 42) produces 10,000 users, 5,000 products in 25 categories, 50,000 transactions and 200,000 browsing sessions as JSON Lines. A validator checks referential integrity and totals; it found the session *converted* flag inconsistent with the transactions (40,607 sessions flagged converted but only 25,204 transactions linked to a session), so in this report **"purchased" means a transaction record exists**.

2. Architecture and technology justification

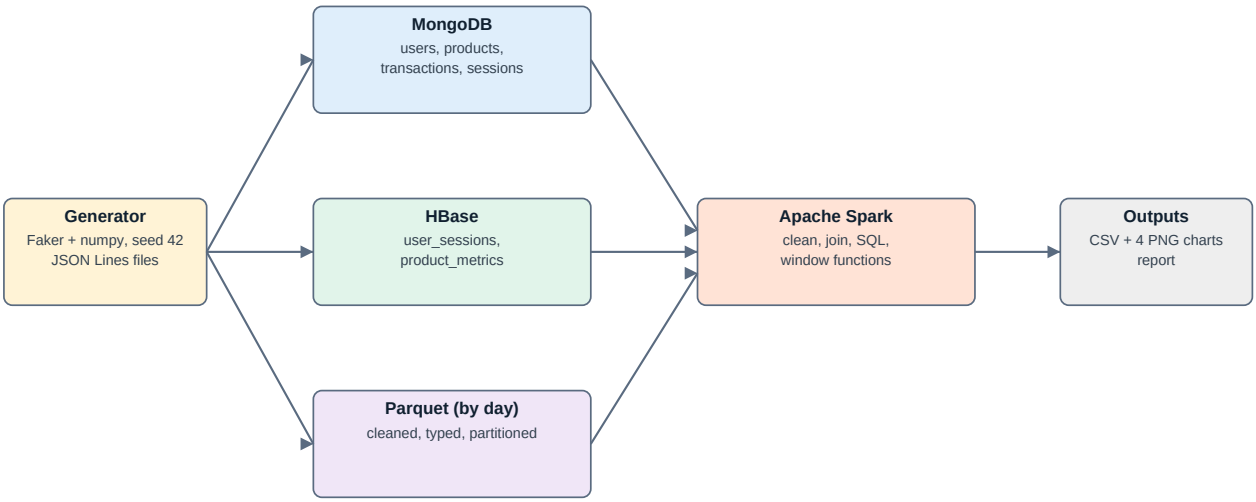


Figure 1. Data flow: the generator writes JSON Lines; the same data is loaded into MongoDB and HBase (each modelled for its own queries) and into Parquet for Spark; the integration step reads HBase and MongoDB into Spark.

System	Used for	Why this system
MongoDB	Users, product catalogue, orders, sessions	Nested, attribute-rich documents; embedded items and category avoid joins; the aggregation framework answers revenue and segmentation questions directly; secondary indexes serve lookups by user and product.
HBase	Per-user session history, per-product daily metrics	Very large, append-only, time-ordered events. One row-key design makes "latest N sessions of a user" a short prefix scan, with no secondary index.
Spark	Cleaning, joins, analytics at scale	Distributed joins and window functions over data that neither store aggregates well; reads Parquet, MongoDB and HBase extracts; scales out by adding executors.

3. Part 1 — Data modelling and storage

3.1 MongoDB document model

- **_id = business key** (user_id, product_id, transaction_id, session_id): no extra id field, free uniqueness.

- **Embedding** where data is read together and bounded: category and subcategory inside products, line items inside transactions, page views inside sessions (the cart dictionary with dynamic product keys is converted to an array so it can be indexed), and a pre-aggregated *purchase_summary* inside each user (total orders and spend, refreshed with bulk updates).
- **Referencing** only across entities that change independently (transactions refer to users and products by id).
- **Indexes:** compound (user_id, timestamp desc) on transactions; multikey on items.product_id and page_views.product_id.

Aggregation pipelines ($unwind, $lookup, $group, $bucket, $dateToString) produce revenue by category and month, user segments by purchase frequency, units sold per product and views per product. Revenue is the sum of item subtotals, before the order-level discount.

Query: latest 5 transactions of one user	Plan	Docs examined	Time
Without the compound index (single-field timestamp index only)	LIMIT > FETCH > IXSCAN (timestamp order, filtered by user)	40,609	83 ms
With the compound index (user_id, timestamp desc)	LIMIT > FETCH > IXSCAN	5	1 ms

The compound index matches the filter and the sort, so MongoDB reads exactly the five documents it returns instead of walking the whole timestamp index — about 8,000 times fewer documents and a large reduction in latency, measured with explain("executionStats").

3.2 HBase schema

Table	Row key	Column families	Access pattern served
user_sessions	user_id#reverse_ts, with reverse_ts = 10^13 – epoch_ms (13 digits, zero-padded, UTC)	info (device, referrer, start, status), pv (page views), cart (cart lines)	Newest-first scan of one user's sessions; time-range scan by computing the two boundary keys; full session detail by one prefix scan.
product_metrics	product_id#yyyymmdd	m (views, cart_adds, purchases)	Daily metrics of one product over a date range.

Reversing the timestamp makes HBase's ascending key order equal newest-first order, so “last N sessions” needs no sort. Putting the user id first keeps one user's data contiguous, and the zero-padded numeric suffix keeps lexicographic and numeric order identical. Three families separate small frequently read attributes from the bulkier event columns, so a query that only needs session summaries never reads page-view cells. A known risk is hotspotting when new users are written in sequence; at scale the user id would be salted or hashed.

4. Part 2 — Spark pipelines

- **Cleaning (spark/clean.py):** explicit schemas (a MapType for the dynamic-key cart), type casts and timestamps, duplicates and inconsistent totals logged, Parquet partitioned by day, row counts reported at each step.
- **Bought-together analysis (spark/analysis.py):** transactions are exploded to items, self-joined to form product pairs, and ranked by support, confidence and lift. With 50,000 transactions, 328 pairs occur at least twice and 83,907 occur only once; high lift values on pairs seen twice are small-count artefacts, so the minimum support threshold matters more than the lift ranking on this data.
- **Spark SQL (spark/sql_queries.py):** (1) monthly revenue by category with a running total and month-over-month growth (window functions: SUM OVER, LAG); (2) top-3 products per category (ROW_NUMBER); (3) conversion by device and referrer; (4) users from a MongoDB-style profile table joined to HBase-style engagement and split into spend quartiles with NTILE (average spend 1,981.95 / 4,106.17 / 5,920.76 / 9,194.83; average sessions per user only 19.2 to 20.8).

Small dimension tables (users, purchases) are joined with broadcast joins to avoid shuffling the large session table. Across the full 200,000 sessions the session-to-purchase conversion is about 12.5–13% in every device and referrer segment. The quartile result shows the synthetic generator makes spend nearly independent of browsing volume: heavy spenders do not browse noticeably more.

5. Part 3 — Integrated analysis: conversion funnel across HBase, MongoDB and Spark

Business question. Where do shoppers drop off between viewing a product and buying, and does it differ by device, referrer and customer purchase frequency?

- HBase supplies the behavioural events (page views, cart) per session — 100,000 sessions scanned in 6 seconds.
- MongoDB supplies the authoritative purchases (25,204 session-linked transactions) and each user's total orders (10,000 users).
- Spark joins the three sources (broadcast joins) and computes cumulative funnel stages: a session counts at a stage only if it passed the earlier ones.

Stage	Sessions	% of sessions	Drop from previous stage
All sessions	100,000	100%	–
Viewed a product page	98,764	98.8%	1.2%
Added to cart	64,987	65.0%	34.2%
Reached checkout	28,979	29.0%	55.4%
Purchased	20,286	20.3%	30.0%

Reconciliation. All 20,286 purchases matched to loaded sessions also passed every earlier stage, which confirms the join keys and stage definitions. The remaining 4,918 session-linked transactions belong to the second session file, which is not loaded in HBase. **Findings.** The largest loss is between cart and checkout (55%). Conversion is flat across devices (20.1–20.4%) and referrers (19.8–21.0%), and rises with customer order frequency from 5.1% (1 order) to 28.8% (7+ orders); because that segment is built from total orders, which include the purchase being measured, the relationship is partly circular and should be read as description, not cause.

6. Engineering notes and data-quality findings

- **Generator defects found and fixed:** the original generator lived in a notebook file and failed outside Jupyter; sessions marked converted outnumbered session-linked transactions; sub-category ids were missing from products. The fixed generator is configurable by environment variables and writes JSON Lines.
- **Reproducibility:** a fixed seed, pinned layout, and a README with the full command sequence; scripts take paths as arguments and print row counts so each step can be checked against the validator.
- **HBase operations:** the Docker Thrift server stopped answering after long idle periods; restarting the container restores it. Scans use column filters and 2,000-row batches so the 100,000-session read finishes in seconds.
- **Spark portability:** on native Windows Spark needs the Hadoop native libraries; the pipeline was verified there and in WSL2/Linux, which is the documented environment.
- **Interpretation guard-rails:** high lift on rarely co-purchased pairs, partial first/last months, revenue before discount and subset mismatches (views from 100,000 sessions, units from all transactions) are stated wherever they affect a number.

7. Part 4 — Visualizations

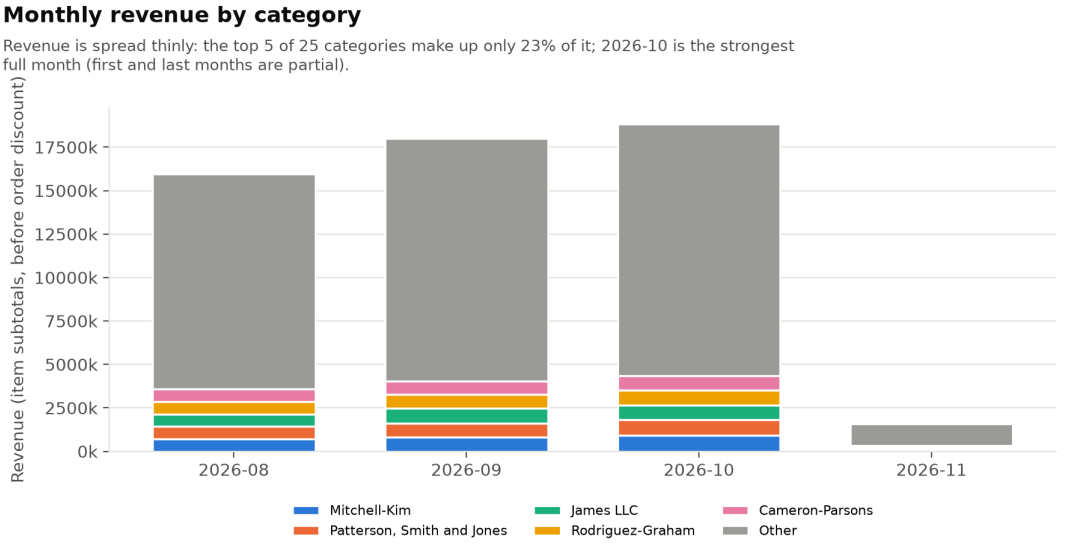


Figure 2. Revenue by category and month. The top 5 of 25 categories make up only 23% of revenue; October 2026 is the strongest full month (August and November are partial months).

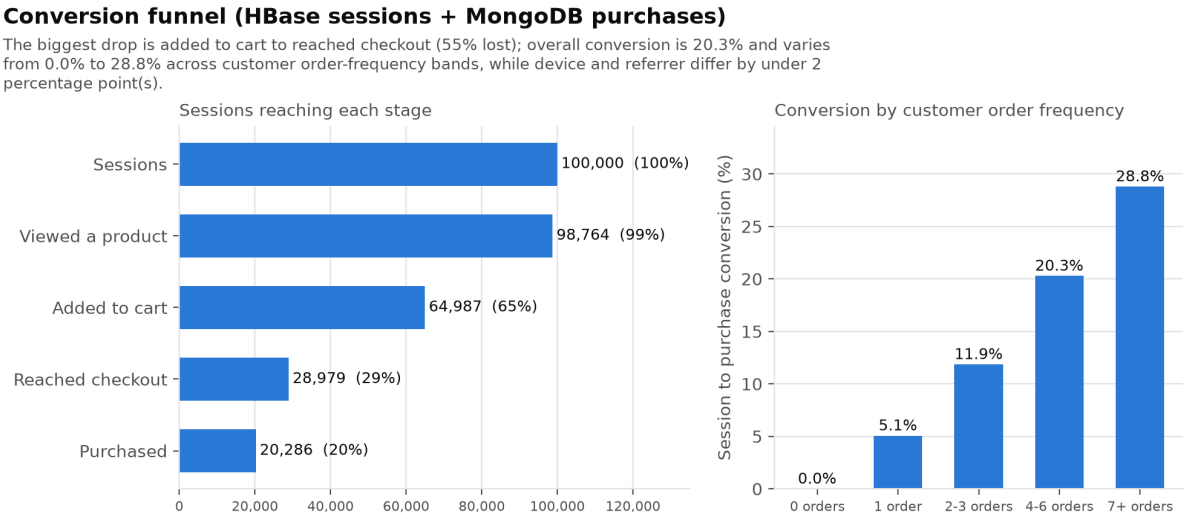


Figure 3. Integrated funnel (HBase + MongoDB). Biggest drop is cart to checkout (55% lost); overall conversion 20.3%; device and referrer differ by under 2 percentage points.

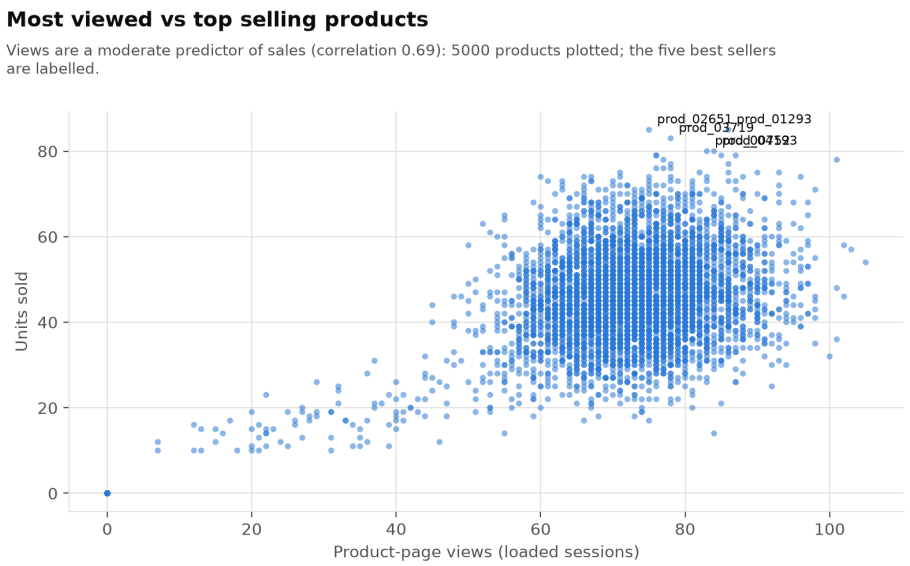


Figure 4. Most viewed vs top selling products (5,000 products). Views are a moderate predictor of units sold (correlation 0.69); views come from the 100,000 loaded sessions while units cover all 50,000 transactions.

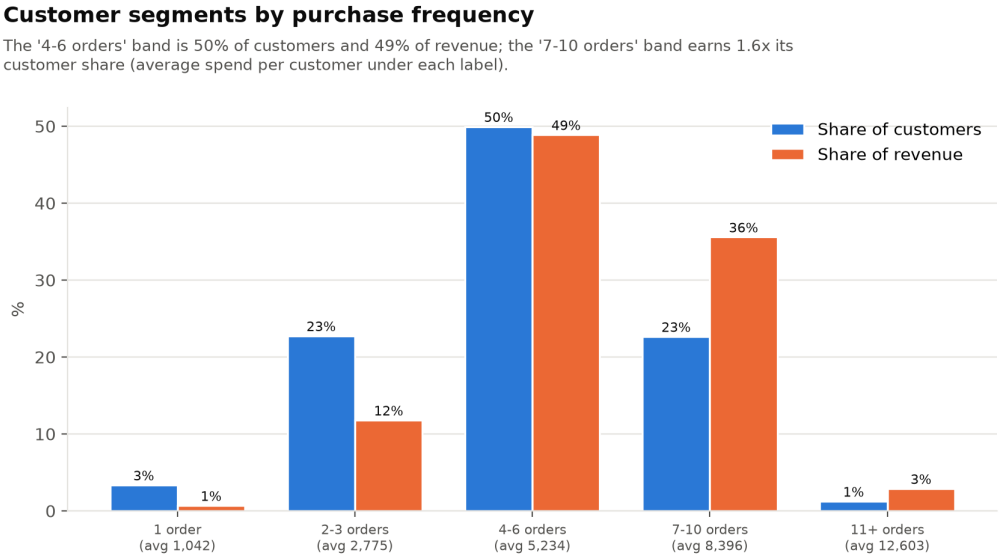


Figure 5. Customers by purchase frequency. The 4-6 orders band is 50% of customers and 49% of revenue; the 7-10 band earns 1.6 times its customer share.

8. Scalability, limitations and conclusion

8.1 Scalability

- **MongoDB:** shard transactions by hashed user_id (even writes, user queries stay single-shard); keep sessions in a time-bucketed or archived collection; replica sets for reads and availability.
- **HBase:** regions split automatically; salt the row-key prefix to avoid hotspotting on new users; keep the number of column families small; set a TTL on old session cells if history can age out.
- **Spark:** the code uses standard DataFrame APIs, so it moves from local mode to a cluster without logic changes; partition Parquet by day, broadcast only small tables, and read HBase and MongoDB through connectors instead of the single-threaded client extracts used here.

8.2 Limitations

- The data is synthetic and largely random, so behavioural signals (device, referrer, browsing volume versus spend) are weak by construction.
- Only the first 100,000 of 200,000 sessions are in MongoDB and HBase. Linked purchases are concentrated in that half (20,286 versus 4,918), so the funnel conversion (20.3%) is higher than the all-sessions figure from Spark SQL (about 12.5–13%); the two use different session subsets.
- Revenue figures are item subtotals before the order-level discount. The first and last months are partial.
- A single-node Docker HBase is for demonstration (its Thrift server timed out after long idle periods and needed a restart); the integration reads through the Python client and a cluster deployment would use native connectors.

8.3 Conclusion

The same retail data was modelled twice, once per access pattern, and combined where each system is weakest: MongoDB gives rich queries on orders and profiles, HBase gives fast time-ordered reads on sessions, and Spark joins them to answer a question neither could alone. The measured index effect (40,609 to 5 documents examined), the validated key design and the reconciled funnel are the main evidence for the design decisions.